

Lesson 2: DOM Manipulation — Selecting and Changing Elements

Mission Brief

The Lumexa Explorer's control panel has real screens now — buttons, readouts, warning lights — but right now they're just static pictures painted onto glass. Today you learn to reach *into* the ship's dashboard from JavaScript and change what's actually displayed: swap text, change colors, add new panels, and remove broken ones. This is the moment your code stops living only in the console and starts controlling the page itself.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what the DOM (Document Object Model) is and how it relates to HTML
- Select single and multiple elements using `document.querySelector` and `document.querySelectorAll`
- Read and change an element's text, HTML content, and CSS styles from JavaScript
- Create new elements and add or remove them from the page
- Add and remove CSS classes dynamically to change appearance

What You'll Build

A "Ship Status Panel" web page: an HTML page with a heading, a status readout, and a list of systems, where a JavaScript file changes the heading text, updates a status message's color based on its value, and dynamically adds new system rows to a list — all without touching the HTML file after page load.

Prerequisites

- Lesson 1 (variables, functions, loops)
- Basic HTML tags (`div` , `p` , `ul` , `li` , `h1`) and basic CSS classes from earlier Lumexa courses

Key Concepts

- The DOM (Document Object Model)
- `document.querySelector()` / `document.querySelectorAll()`
- `.textContent` vs `.innerHTML`
- `.style` property and inline styles
- `classList.add()` / `.remove()` / `.toggle()`

- `document.createElement()` and `.appendChild()` / `.append()`
- `.remove()` for deleting elements

Concept Explanation

When a browser loads an HTML page, it doesn't just display the text — it builds an internal, live, tree-shaped model of every element called the **DOM (Document Object Model)**. Each HTML tag becomes a **node** in this tree, and JavaScript can read and modify that tree at any time, and the browser instantly re-renders the page to match. This is the mechanism behind every dynamic website you've ever used.

To modify an element, you first need to **select** it. `document.querySelector(selector)` returns the *first* element matching a CSS selector (like `"#status"`, `".system-row"`, or `"button"`), or `null` if nothing matches. `document.querySelectorAll(selector)` returns *all* matching elements as a `NodeList` (which behaves like an array — you can loop over it with `for` or `.forEach()`).

Once you have a reference to an element, you can change what's inside it. `.textContent` sets or reads the plain text inside an element — safe and simple. `.innerHTML` sets or reads the actual HTML markup inside an element, which lets you insert nested tags, but is more powerful (and riskier — inserting unsanitized user input with `.innerHTML` can create security vulnerabilities, so prefer `.textContent` unless you specifically need to inject HTML).

You can change how an element looks with the `.style` property (e.g., `element.style.color = "red"`) for one-off inline styles, but the better practice for anything reusable is to define CSS classes in your stylesheet and toggle them with `.classList.add("className")`, `.classList.remove("className")`, or `.classList.toggle("className")`. This keeps your styling logic in CSS, where it belongs, and your JavaScript simply flips switches.

Sometimes you need to build entirely new elements — for example, adding a new to-do item or a new system row. `document.createElement("tagName")` creates a new, detached element in memory. You then set its content/attributes, and finally attach it to the visible page with a parent element's `.appendChild(newElement)` or the newer `.append(newElement)` method. To remove an element entirely, call `.remove()` directly on it.

Guided Coding

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Ship Status Panel</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; }
    .status-ok { color: #4ade80; font-weight: bold; }
    .status-warning { color: #facc15; font-weight: bold; }
    #systems-list li { padding: 0.25rem 0; }
    .new-system { border-left: 3px solid #60a5fa; padding-left: 0.5rem; }
  </style>
</head>
<body>
  <h1 id="ship-heading">Lumexa Explorer</h1>
  <p>Status: <span id="status-readout">Unknown</span></p>
  <ul id="systems-list">
    <li>Life Support</li>
    <li>Navigation</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

```

// script.js

// --- SELECTING ELEMENTS ---
const heading = document.querySelector("#ship-heading");
const statusReadout = document.querySelector("#status-readout");
const systemsList = document.querySelector("#systems-list");

// --- CHANGING TEXT ---
heading.textContent = "Lumexa Explorer – Systems Online";

// --- CONDITIONAL STYLING BASED ON DATA ---
const fuelLevel = 18; // percent, deliberately low to trigger a warning

if (fuelLevel < 20) {
  statusReadout.textContent = "LOW FUEL WARNING";
  statusReadout.classList.add("status-warning");
} else {
  statusReadout.textContent = "Nominal";
  statusReadout.classList.add("status-ok");
}

// --- CREATING AND APPENDING NEW ELEMENTS ---
const newSystems = ["Communications", "Docking Clamps", "External Cameras"];

newSystems.forEach((systemName) => {
  const listItem = document.createElement("li");
  listItem.textContent = systemName;
  listItem.classList.add("new-system");
  systemsList.appendChild(listItem);
});

// --- SELECTING MULTIPLE ELEMENTS ---
const allListItems = document.querySelectorAll("#systems-list li");
console.log(`There are now ${allListItems.length} systems listed.`);

allListItems.forEach((item, index) => {
  console.log(`System ${index + 1}: ${item.textContent}`);
});

// --- REMOVING AN ELEMENT ---
// Suppose "Navigation" is offline for maintenance and must be removed.
allListItems.forEach((item) => {
  if (item.textContent === "Navigation") {
    item.remove();
  }
});

```

Code Walkthrough

- **Selecting elements:** `querySelector("#ship-heading")` uses a CSS ID selector to grab exactly one element. Storing it in a `const` lets us reuse the reference without re-querying the DOM every time.
- **Changing text:** setting `.textContent` directly replaces whatever plain text was inside the `<h1>`.
- **Conditional styling:** rather than hardcoding a style, the code checks a real value (`fuelLevel`) and applies one of two CSS classes already defined in the `<style>` block. This is the recommended pattern: JavaScript decides *which* class applies, CSS decides what that class *looks like*.
- **Creating elements:** `document.createElement("li")` builds a new list item in memory only — it isn't visible yet. `.textContent` fills it in, `.classList.add()` styles it, and `.appendChild()` finally attaches it to the visible `<ul>`, one per array item via `.forEach()`.
- **Selecting multiple elements:** `querySelectorAll` returns a `NodeList` of every `<li>` inside `#systems-list`, including the ones we just added — proof that the DOM updates live. `.forEach()` iterates it just like an array.
- **Removing an element:** rather than searching by ID, this loops through all items and calls `.remove()` on whichever one's text matches — demonstrating how you can find and delete elements based on data, not just fixed selectors.

Student Mission

1. Add a button `<button id="add-system-btn">Add Random System</button>` to the HTML.
2. In `script.js`, select the button and write code so that, when the page loads, three new randomly-named systems are appended (don't worry about click events yet — that's Lesson 3; for now, just have the array/loop logic run automatically on page load).
3. Add a new CSS class `.critical` (red background, white bold text) and apply it via `classList.add()` to any system whose name includes the word `"Docking"`.

Challenge

Write a function `renderSystemsList(systemNames, containerSelector)` that takes an array of names and a CSS selector string, clears out any existing children of that container, and rebuilds the list from scratch using `createElement` / `appendChild`. This "clear and re-render" pattern is the foundation of how real front-end frameworks update the screen.

Experiment/Extension

Open DevTools, go to the **Elements** tab, and watch the actual HTML update live as your script runs. Try running `document.querySelector("#ship-heading").style.color = "hotpink"` directly in the Console and watch the page change instantly — this proves the DOM is a live, editable structure, not just a static file.

Common Mistakes

- Calling `document.querySelector` before the HTML elements exist in the page (e.g., placing your `<script>` tag in the `<head>` without `defer`), which returns `null` and causes `Cannot set properties of null` errors.
- Confusing `.textContent` (safe, plain text) with `.innerHTML` (parses HTML, riskier with untrusted input).
- Forgetting that `querySelectorAll` returns a static `NodeList`, not a live array — reassigning it after adding elements requires re-querying.
- Using `.style.color` directly everywhere instead of toggling CSS classes, which quickly becomes hard to maintain.

Debugging Tips

- If you get `Cannot read properties of null (reading 'textContent')`, your selector didn't match anything — check the Elements tab for typos in your ID/class name, or make sure the script runs after the HTML has loaded (place `<script>` at the end of `<body>`, as in this lesson).
- Use the Elements tab's search (Ctrl+F inside it) to confirm an element with your expected ID actually exists.
- Right-click any element on the page and choose "Inspect" to jump straight to its DOM node in the Elements panel.
- Add `console.log(heading)` right after selecting it — if it prints `null`, you know the selection failed before anything else goes wrong.

Check Your Understanding

1. What is the DOM, in your own words?
2. What's the difference between `querySelector` and `querySelectorAll`?
3. Why is `.textContent` generally safer than `.innerHTML`?
4. What two steps are required to add a brand-new element to the visible page?
5. Why is toggling a CSS class usually better than setting `.style` properties directly?

Mini Quiz + Answer Key

Q1. Which method selects *all* matching elements? A) `document.querySelector` B) `document.querySelectorAll` C) `document.getElement` D) `document.select`

Q2. What must happen before a newly created element with `createElement` appears on the page? A) Nothing, it appears automatically B) It must be appended to a parent element already in the DOM C) You must reload the page D) You must call `.render()`

Q3. Which property directly sets the plain text inside an element? A) `.innerHTML` B) `.value` C) `.textContent` D) `.className`

Q4. What does `classList.toggle("active")` do if the element already has the `active` class? A) Adds it again
B) Removes it C) Throws an error D) Does nothing

Answer Key: 1-B, 2-B, 3-C, 4-B

Lesson Recap

You learned that the DOM is a live, editable tree representation of the page, and practiced selecting elements with `querySelector` / `querySelectorAll`, changing their text and styling, creating brand-new elements, and removing existing ones — the core toolkit for building any interactive interface.

Homework

Build a small "Crew Roster" page with an empty `<ul id="roster">`. Write a script that defines an array of at least 5 crew objects (`{ name, role }`), loops over it, and for each one creates an `<li>` containing the crew member's name and role (e.g., `"Ada – Pilot"`), appending each to the roster list.

Portfolio Project Connection

DOM manipulation is the visual engine behind every project in this course: rendering to-do items to the screen (Project 04), displaying fetched weather data in readable panels (Project 05), and rendering the flip-able card grid for the memory game (Project 06) all depend directly on the selecting, creating, and updating techniques from this lesson.